Table des matières

Comment fonctionne l’application:.....1
L’état de l’avancement:.....2
Les améliorations éventuelles:.....3
L’organisation du programme:.....3
Les choix techniques du programme:.....4
Les éventuels problèmes rencontrés:.....5
Répartition du travail:.....6

Comment fonctionne l’application:

Pour réaliser l’interface graphique nous avons utilisé la librairie « ZEN 6 ». Il est donc important d’importer cette librairie ainsi que le fichier « deck.txt » qui recense toutes les cartes du jeu. Pour lancer le jeu, il suffit d’exécuter le fichier nommé « GameController.java ».

Menu de démarrage :

Lorsque le programme est lancé, un menu s’affiche pour sélectionner le nombre de joueurs. Il est donc possible de jouer tout seul, à 2 joueurs, à 3 joueurs ou bien à 4 joueurs. Les options sont représentées sous la forme de boutons cliquables.

Déroulement du programme :

La boucle gameloop gère le déroulement du jeu pour chaque joueur. Si le joueur gagne, un message est affiché et la fenêtre se ferme.

La gestion des clics de souris des touches du clavier permet de sélectionner, placer, piocher différentes cartes. Il est aussi possible de se déplacer dans la zone de jeu avec les touches directionnelles. La touche échap permet de quitter le jeu.

Dès chaque tour, chaque événement (pose de cartes, pioche,…) nous effaçons la zone de jeu pour la redessiner.

Exemple d’utilisation :

- Lancer le programme
- Choisir le nombre de joueurs
- Interagir avec la zone de jeu (pose des cartes, pioche, verso, recto, déplacement,…)
- Si un joueur gagne le programme se ferme automatiquement

Fonctionnement des touches :

Pour sélectionner une carte dans notre main, il suffit d'appuyer soit sur la touche w,x,c. Lorsqu'on a sélectionné une carte on peut, cliquer sur un coin d'une carte déjà placer, on peut aussi regarder le verso de la carte grâce à la touche A. Pour regarder le recto on appuie sur la touche Z. Dès que nous avons posé une carte nous pouvons ensuite piocher parmi les 6 cartes (les 2 pioches côté verso, 2 cartes dorures et 2 cartes ressources affichées) de la pioche située en haut à droite. Il suffit juste de cliquer sur la carte que nous voulons piocher.

L'état de l'avancement:

Parties terminés du projet :

Phase 1 :

Pour la première phase du rendu nous avons implémenté la ressource « Animal ». Pour le décompte des ressources nous avions le système d'accumulation des ressources. Pour le gain de point nous avions implémenté le scoring « direct » ainsi que le décompte du nombre total de points gagnés. Pour la partie carte, nous avions réussi à afficher les cartes ressources ainsi que les cartes dorure. Nous avions ajouté tous les types de coins (coins visibles, invisibles, avec des ressources). Pour la partie table de jeu, nous avions la structure de données pour la table de jeu, la pose d'une carte. Pour la partie main du joueur nous avions une pioche ainsi que la pose d'une carte. Pour l'affichage nous avions affiché les cartes, la table du joueur avec sa main et son score. Enfin pour la qualité du code. Nous avions découpé notre code en classes, interfaces, polymorphisme. Les méthodes sont de taille raisonnable exceptée une méthode.

Phase 2 :

Pour la seconde phase du rendu nous avions implémenté les cartes de départ, les artefacts (tous les types possibles : plume, parchemin, encrier). Nous avions implémenté tous les types de ressources (végétal, champignon, insecte), le verso des cartes, le système de gain de point par recouvrement de coins des cartes, l'affichage des ressources sur les cartes. Enfin une interface plus complète qui permet d'afficher le verso des cartes au-dessus de la pioche.

Phase 3 :

Pour la dernière phase du rendu nous avons implémenté le mode multijoueur qui nous a pris une grande partie du temps. Nous avons aussi implémenté les deux cartes ressources et les deux cartes dorures en permanence dévoilées. Mais encore un système permettant de charger un deck à partir d'un fichier texte. Nous avons amélioré l'interface graphique (ajout d'un menu permettant le choix du nombre de joueurs, ajout de quelques textes pour connaître les différentes touches pour interagir avec le jeu). Par ailleurs, nous avons implémenté un système de déplacement dans la zone de jeu. Enfin nous avons implémenté le coût pour pouvoir poser les cartes dorures.

Les améliorations éventuelles:

Phase 2 :

La première amélioration que nous avons réalisée pendant la phase numéro 2 est la pose de carte complète. Lors de la première phase il été possible de poser une carte même si un coin n'était pas libre. Ce problème a été donc corrigé durant la phase 2. Ensuite nous avons amélioré l'affichage des différents types de score sur les cartes dorures. Auparavant nous n'avions affiché que le score de type DirectScoring. Ce problème a également été corrigé durant la phase 2.

Phase 3 :

La première amélioration que nous avons réalisée pendant cette période est l'ajout du système de gain de point associé en fonction des ressources visibles. Nous n'avions malheureusement pas eu le temps de le réaliser dans la phase 2. Ensuite nous avons amélioré la qualité du code notamment en ajoutant tous les Objects.requireNonNull, les commentaires mais encore la diminution de la taille de certaines méthodes.

L'organisation du programme:

Le code est réparti en plusieurs classes et records :

La première classe (AreaGame) permet de définir toute la zone de jeu où il est possible de poser les différentes cartes.

Ensuite le record Choice a pour but de changer le score de la carte en fonction des gains de points par recouvrement des coins.

Le record Coordinates permet d'encapsuler les deux coordonnées x et y.

La classe Corner recense toutes les méthodes en corrélation avec les coins d'une carte, notamment si le coin possède une ressource, s'il est libre ou non pour pouvoir poser une carte par-dessus.

La classe GameBoard contient la gestion de la main de carte notamment la main de départ en piochant les cartes, les méthodes permettant de piocher une carte ressource et dorure si la main du joueur n'est pas pleine. Ensuite la classe contient la gestion du choix de la carte, notamment la mise à jour du choix de la carte en fonction de la touche pressée par le joueur. Mais encore la classe contient la gestion de l'aire de jeu notamment la fonction de déplacement dans la zone de jeu, le placement de la carte choisie dans l'aire de jeu si les conditions sont remplies. Par ailleurs, la classe comprend le calcul et la gestion des scores notamment avec l'ajout du score d'une carte dans une liste des scores, le calcul du score total du joueur et la vérification si le joueur a atteint le score de 20 points pour gagner les scores de la partie. Enfin la classe contient le comptage des ressources qui comprend la mise à jour des comptages des ressources en fonctions des cartes jouées et retourne les compteurs de façon non-mutable afin de ne pas pouvoir modifier les compteurs.

La classe GameController permet de lancer le programme.

Le record GameView permet de dessiner toutes les informations visibles sur l'écran du joueur. Cela comprend donc les affichages de tous les types de cartes, tous les types de scores, les pioches ainsi que la pioche du joueur.

La classe GoldCard recense de nombreuses méthode notamment le système de coût pour poser la carte, les différents type de compteur pour le score ainsi que la possibilité de poser une carte dorure.

La classe RessourceCard possède quasiment les mêmes méthodes que la classe GoldCard.

La classe Score comprend toutes les méthodes permettant de déterminer tous les types de score.

La classe Stack liste les cartes ressources et dorures. Elle permet de générer une carte dorure et ressource cachée ainsi que les deux cartes dorures et ressources visibles.

La classe StarterCard représente la carte de départ, elle comprend différentes propriétés comme face (recto/verso) et les coins de cartes.

L'interface Card. Nous avons ajouté une interface pour pouvoir utiliser le polymorphisme.

Les choix techniques du programme:

Les méthodes fondamentales pour pouvoir jouer correctement sont :

La méthode placeCard(PointerEvent pointerEvent, ApplicationContext context) permet de placer une carte. Cette méthode vérifie si la carte que l'on souhaite placer à l'endroit que l'on a cliqué respecte toutes les conditions nécessaires comme la disponibilité de l'emplacement et si la carte possède un coin disponible. Ensuite, si les conditions sont remplies, la carte sélectionnée est placée sur le plateau. Puis la zone de jeu est redessinée pour réactualiser la main, etc. Enfin, la méthode met à jour les compteurs de ressources.

La méthode totalScore() cette méthode permet de calculer le score total du joueur permettant de déterminer la fin de la partie. Cette méthode additionne les scores de toutes les cartes pour obtenir le score total, il est ensuite utilisé pour vérifier si le joueur a atteint les conditions de victoire.

La méthode startHand(Stack stack) initialise la main de départ du joueur pour commencer le jeu. Cette méthode ajoute des cartes ressources et dorures à la main du joueur depuis le stack.

Pour représenter notre jeu de jeu nous avons décidé d'utiliser une HashMap<Coordonnées,Cartes> afin de lier directement les cartes avec leur coordonnées respective. De plus on a une ArrayList<Coordonnées> checkcoor qui permet de de trouver quelle coordonnées d'une carte de la zone de jeu correspondent aux coordonnées du clic de la souris s'il y a une carte (methode isCard). La méthode isCard ajoute en dernière position de la liste checkcoor. Ainsi après un passage en boucle dans la HashMap pour trouver les coordonnées correspondantes, la liste checkcoor permet de reduire en complexité (et de calculer le score par recouvrement de coins en bonus).

Les éventuels problèmes rencontrés:

- La taille d'une certaines méthodes
- On a pas réussi à importer d'image dans le jeu.
- Un problème qui nous a suivis pendant tout le projet, lorsque le jeu s'arrête après une victoire il y avait une exception avec le context.dispose();.

- Les cartes objectifs qui ne sont pas présentes dans le jeu car, on ne trouvait pas comment faire pour les objectifs avec un pattern a suivre.

Répartition du travail:

Phase 1 :

- Les cartes ressource et dorure, avec un seul type de ressource, la ressource “Animal”. On peut de plus supposer que la ressource s’accumule : autrement dit, si l’on recouvre une ressource, on ne la perd pas : **Antonin / Daniel**
- Le gain de point “direct”, c’est-à-dire lors de la pose d’une carte : **Antonin**
- Le jeu en solo, qui consiste à atteindre 20 points en le moins de tours possibles : **Antonin**
- Une interface graphique qui permet d’afficher la zone de jeu d’un joueur et d’y jouer des cartes, ainsi que la main du joueur : **Daniel**

Phase 2 :

- Les artefacts (plume, parchemin, encrier) : **Antonin**
- Le système de gain de points associé : **Antonin**
- Le système de gain de points par recouvrement de coins : **Antonin**
- Les cartes de départ : **Daniel**
- Le verso des cartes : **Daniel**
- Une interface graphique plus complète, qui permet notamment d’afficher le verso de la carte située en haut de la pioche : **Daniel**

Phase 3 :

- Le mode multijoueur : **Daniel**
- En plus de la pioche, il y a en permanence deux cartes ressource et deux cartes dorure dévoilées, que les joueurs peuvent choisir plutôt que de piocher : **Antonin / Daniel**
- Une interface graphique complète : **Daniel**
- Un système permettant de charger un deck : **Antonin / Daniel**

Daniel : 55%

Antonin : 45%